

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

CURSO INTEGRADOR II: SOFTWARE (100000S12F)

DOCUMENTO DE ARQUITECTURA DE SOFTWARE (SAD)

MODELO DE VISTAS 4+1 DE KRUCHTEN Y ESTÁNDAR C4 (LEVEL 2)

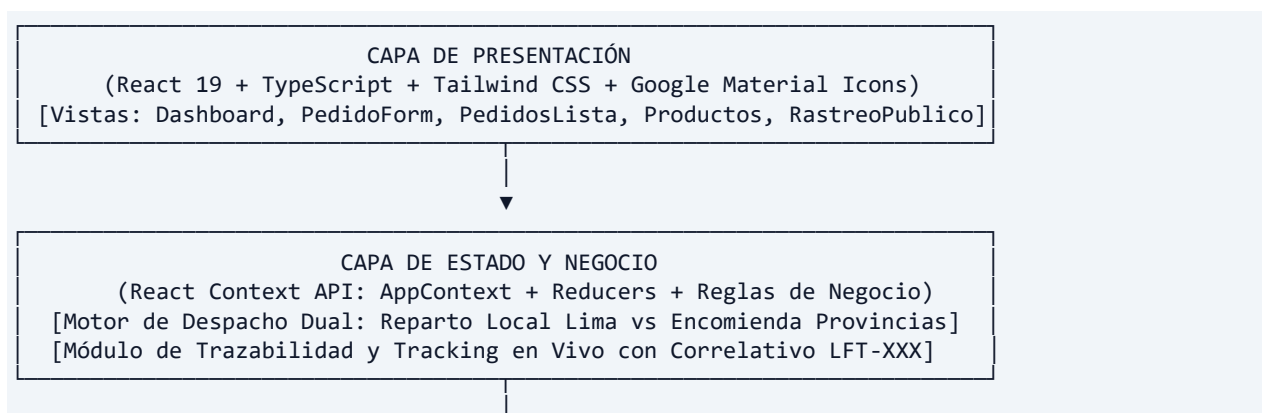
PROYECTO: SISTEMA WEB PWA DE GESTIÓN DE PEDIDOS, ENVÍOS LOCALES Y ENCOMIENDAS NACIONALES PARA LEOFIT

1. CONTROL DEL DOCUMENTO Y EQUIPO DE ARQUITECTURA

- **Institución:** Universidad Tecnológica del Perú (UTP)
- **Curso:** Curso Integrador II: Software (100000S12F)
- **Proyecto:** Progressive Web App (PWA) de Gestión de Pedidos & Inventario LeoFit
- **Equipo de Desarrollo (Grupo 01):**
 1. **Loayza Rodriguez, Lady Luz** - Scrum Master / UX-UI Lead
 2. **Cárdenas Fernández, Víctor Leandro** - Product Owner / Arquitecto Back-End y Base de Datos
 3. **Roman Delgado, Harley Anthony** - Front-End Lead / Especialista PWA y WPO
 4. **Dávila Morales, Jim Alessandro** - QA Engineer / Automatización de Pruebas
 5. **Rojas Sanchez, Daniel Enrique** - Analista Funcional / Modelado de Procesos
- **Versión del Documento:** 1.2.0

2. REPRESENTACIÓN ARQUITECTÓNICA (VISTA LÓGICA Y DESACOPLAMIENTO)

El sistema implementa una **Arquitectura Limpia (Clean Architecture)** organizada en capas concéntricas con flujo de dependencias unidireccional y motor de despacho dual:



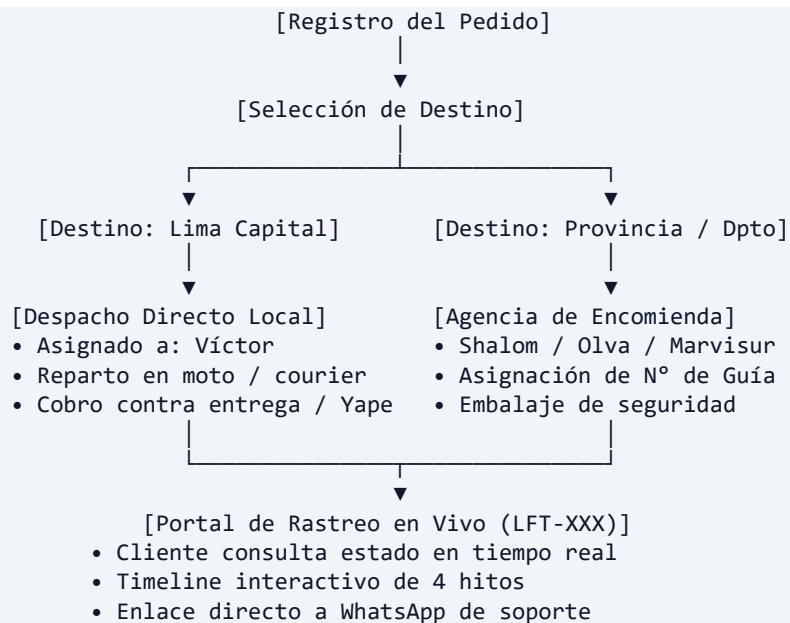
▼

CAPA DE ACCESO A DATOS
(Motor de Persistencia en Memoria / Cache Offline LocalStorage /
Integración con APIs de Mensajería WhatsApp wa.me)

3. VISTA DE DESARROLLO (ESTRUCTURA MODULAR)

```
frontend/src/
├── __tests__/           # Suite de pruebas unitarias automatizadas (Vitest)
├── components/          # Componentes modulares reutilizables
│   ├── common/         # Componentes atómicos (Badge, Modal, MontoPrivado)
│   └── layout/          # Elementos estructurales (Navbar con menú desplegable)
├── context/             # Capa de estado global inmutable (AppContext.tsx)
├── data/                # Modelos de datos TypeScript y catálogo (mockData.ts)
├── pages/               # Vistas de la aplicación:
│   ├── Dashboard.tsx   # KPIs financieros, alertas en riesgo y pipeline
│   ├── PedidosLista.tsx # Historial con soporte de guías de encomienda
│   ├── PedidoForm.tsx  # Registro de pedidos (Lima vs Provincias Shalom/Olva)
│   ├── ProductosGestion.tsx # Catálogo de indumentaria y alertas de stock
│   ├── RastreoPublico.tsx # Portal de tracking en vivo para clientes finales
│   └── Login.tsx       # Control de acceso y sesión segura de Víctor
```

4. VISTA DE PROCESOS (FLUJO DE DESPACHO DUAL Y RASTREO)



5. VISTA DE CONTENEDORES (C4 MODEL LEVEL 2)

El diagrama de contenedores formal documenta los siguientes límites de sistema:

6. **Usuario Administrador (Víctor):** Opera la PWA para registrar ventas, gestionar inventario y asignar números de guía de encomienda.

7. **Cliente Final:** Realiza compras por WhatsApp y consulta el estado de su envío en el Portal de Rastreo Público.
8. **PWA Frontend (React 19 + TypeScript):** Interfaz reactiva con modo accesible de alto contraste y compatibilidad offline.
9. **Backend Hub:** Motor de transacciones, cálculo de fletes y correlativos **LFT-XXX**.
10. **Sistemas Externos de Logística:**
 - **WhatsApp Gateway:** Notificaciones y confirmación de delivery local.
 - **Agencias de Encomienda (Shalom / Olva Courier / Marvisur):** Transporte interprovincial a nivel nacional.

6. VISTA FÍSICA Y DE DESPLIEGUE (EDGE SERVERLESS CDN)

- **Proveedor de Infraestructura:** GitHub Pages / Vercel Serverless Edge Network.
- **Seguridad y Transporte:** Certificado SSL/TLS nativo (HTTPS forzado).
- **Distribución de Contenidos:** Red de Entrega de Contenidos (CDN) global con puntos de presencia de baja latencia.
- **Integración Continua:** GitHub Actions ejecutando pipelines automáticos de auditoría de seguridad y despliegue continuo ante cada confirmación a la rama **main**.